

# Fix-Brief: Versetzte Versandlabel

Übergabe an Claude Code · Ursachenanalyse, Entscheidung, Patches, Testplan · Stand: Juni 2026

## Auftrag an Claude Code (Kurzfassung)

Behebe die **versetzte Auftrags-/Label-Zuordnung**: Wenn mehrere Pakete mit demselben EAN hintereinander durch die Maschine laufen, bekommt ein Paket das Versandlabel eines *anderen* Auftrags. Konkret:

- Die drei Patches in Abschnitt 6 umsetzen ( `connection.py` : ENQ-Block, `_claim_pulpo_order` , `_precreate_label` ).
- Die Edge-Cases in Abschnitt 7 berücksichtigen (Resume, Nebenläufigkeit, FAILED-Guard, Orphan-Label).
- Die Akzeptanztests in Abschnitt 8 als `pytest` -Tests implementieren und grün bekommen.
- Zum Schluss eine Vergleichs-Markdown `docs/cmc-orderbinding-fix.md` schreiben (Vorher/Nachher + Anti-Patterns, Abschnitt 9).

**Nicht** die Protokoll-Logik, die CW-Listen-Mengenprüfung oder die DHL-Anbindung umbauen — nur den Zeitpunkt und die Persistenz der Auftragsbindung.

## 1. System-Kontext

Die Software ist eine CIS-Middleware (das „Gehirn“) zwischen vier Partnern: dem Pulpo WMS (Packaufträge, Adressen), der DHL-API (Label-Erzeugung), der CMC CartonWrap-Maschine (Protokoll) und dem Labeldrucker. Jedes physische Paket durchläuft die Stationen in fester Reihenfolge, an jeder Station tauscht die Maschine eine Protokoll-Nachricht aus:

ENQ (Scan) → IND (Induktion) → ACK (3D-Vermessung) → LAB1 (Label) → Druck → END  
(Exit-Prüfung)

**Korrelation:** Bei ENQ sendet die Maschine nur einen Event-Zähler; die Cloud vergibt daraus die `reference_id` ( `ref0014` ) und spiegelt sie zurück. Bei IND/ACK/LAB1/END echot die Maschine diese `reference_id` mit. Tracker und Auftragsbindung hängen an ( `protocol_id` , `reference_id` ). Mehrere Pakete sind gleichzeitig an verschiedenen Stationen unterwegs.

## 2. Symptom

### BEOBACHTET

Drei Bestellungen hintereinander durchlaufen lassen → die Labels waren **versetzt**: falsches Label pro Artikel bzw. falsche Auftragsnummer auf dem Paket. „Versetzt“ (nicht zufällig) ist der

entscheidende Hinweis — Paket 1 bekommt das Label von Auftrag B, Paket 2 das von A usw. Das ist eine **Queue-Desynchronisation**, kein Einzelfehler.

### 3. Ursachenanalyse

#### Hauptursache — Bindung in einem nicht-awaited Hintergrund-Task

Die Zuordnung Paket → konkreter Pulpo-Auftrag ( `_claim_pulpo_order` ) wird *zuerst* in `_precreate_label` ausgeführt. Diese Funktion wird bei IND als **Fire-and-forget** gestartet:

```
asyncio.create_task(self._precreate_label(conn.protocol_id, ref_ind, dict(msg_data)))  
# nicht awaited
```

Damit hängt die Reihenfolge, in der die Pakete ihren Auftrag binden, **nicht an der physischen Scan-Reihenfolge**, sondern daran, welcher Hintergrund-Task im Event-Loop zuerst durch seine DB-Query ( `await db.execute(...)` ) kommt. Weil `_claim_pulpo_order` „den ersten freien Auftrag in FIFO-Reihenfolge nach `created_at`“ nimmt, schnappt sich der Task, der zufällig zuerst fertig wird, den ältesten Auftrag — egal zu welchem Paket er gehört.

#### FOLGE

Bei mehreren gleichen EANs ist die Bindung effektiv zufällig verwürfelt. Sobald es einmal verschoben ist, **kaskadiert** es, weil jeder Folge-Claim „den nächsten freien“ Auftrag greift.

#### Zweite Fehlerquelle — der defensive Fallback

Am Ende von `_claim_pulpo_order` steht ein Notnagel, der bei „mehr Scans als Aufträge“ defensiv den ersten Auftrag bindet:

```
# Mehr Scans als Aufträge → defensiv den ersten nehmen.  
bound_map[ref] = rows[0].id  
return rows[0]
```

Das bindet eine möglicherweise **bereits belegte** Order an eine zweite `ref` → garantiert ein falsches Label statt einer sichtbaren Ablehnung. Ein stiller Fehler ist schlimmer als ein sauberes Reject.

#### Dritte Schwäche — flüchtige Reservierung

Der Reservierungs-Guard liest nur die In-Memory-Map `self._ref_pulpo_order`:

```
claimed = {oid for r, oid in bound_map.items() if r != ref}
```

Diese Map ist nicht persistiert. Sie übersteht keinen Neustart, und sie kennt den `FAILED`-Zustand nicht. Ein bereits gelabeltes Paket, dessen Pulpo-Abschluss fehlschlug ( `FAILED` ), kann dadurch erneut beansprucht werden → **Doppel-Sendung**.

## 4. Vergleich mit der funktionierenden Vorlage

Die mitgelieferte Vorlagen-Doku (4Fulfillment, „Process Documentation“) beschreibt ein gleichartiges, aber funktionierendes System. Der entscheidende Unterschied steht dort wörtlich in §7.2 „*Order Reservation at ENQ*“ und §4 (State-Lifecycle): dort wird der Auftrag **bei ENQ** gebunden und sofort als **ASSIGNED**-State persistiert.

| Aspekt                             | Funktionierende Vorlage                       | Eure Implementierung (vorher) | Fix                                        |
|------------------------------------|-----------------------------------------------|-------------------------------|--------------------------------------------|
| Bindungs-Zeitpunkt                 | ENQ — sofort, synchron, State <b>ASSIGNED</b> | IND — async, nicht awaited    | ENQ — synchron, awaited                    |
| Reservierungs-Quelle               | Persistierte aktive State-Dokumente           | Flüchtiges In-Memory-Dict     | Persistierte aktive OrderStates            |
| FAILED im Guard                    | Ja (verhindert Doppel-Sendung)                | Nein                          | Ja                                         |
| Überzahl (mehr Scans als Aufträge) | Am Scanner ablehnen                           | Falsche Order binden          | Ablehnen ( <b>None</b> → <b>result=0</b> ) |
| Neustart-Sicherheit                | Ja (State in DB)                              | Nein (Map weg)                | Ja                                         |

## 5. Entscheidung & Leitprinzip

### LEITPRINZIP

Ein physisches Paket = eine **reference\_id** = eine **pulpo\_order\_id**. Die Bindung wird im **frühesten sicheren Moment (ENQ)** hergestellt, **synchron** (awaited), **sofort persistiert** (State **ASSIGNED**) und unverändert bis END getragen. Gejoint wird überall über (**protocol\_id**, **reference\_id**) — nie über Barcode-FIFO neu abgeleitet. Precreate und LAB1 **lesen** die Bindung nur.

Der Kern: Slot-Abbuchung und Auftragsbindung müssen **ein und derselbe Schritt** sein, ausgeführt in Scan-Reihenfolge. Beides liegt bei ENQ bereits vor ( **ref** vergeben, Barcode im Tracker, Slot wird hier abgebucht) — der Claim wurde nur ohne Not in den async Precreate verschoben.

## 6. Die drei Patches

### 1 ENQ — synchron claimen und persistieren

Im Read-Loop, im bestehenden Slot-Block (`connection.py`). Der Claim wandert an die Stelle, wo der Slot abgebucht wird:

```
if msg_type == "ENQ" and conn.protocol_id and response.get("result") == 1:
    bc = str(msg_data.get("barcode", "") or "").strip()
    ref = str(response.get("reference_id") or "")
    if matched_cw_list and filter_passed and not is_resume:
        if self.consume_cw_entry(conn.protocol_id, matched_cw_list, bc):      #
consumed +1
            self.record_cw_consumption(conn.protocol_id, ref, matched_cw_list, bc)
            # NEU: Auftrag SOFORT und AWAITED binden – Scan-Reihenfolge = Bindungs-
Reihenfolge
            async with async_session() as db:
                tenant_id = await self._tenant_for(db, conn.protocol_id)
                claimed = await self._claim_pulpo_order(db, conn.protocol_id,
tenant_id, ref, bc)
                if claimed is None:
                    # kein freier Auftrag → NICHT akzeptieren (sonst Doppelbindung →
versetzt)
                    self.release_cw_for_ref(conn.protocol_id, ref)            # Slot
zurückgeben

                    response["result"] = 0
                    response["item_validated"] = False
                    response["rejection_reason"] = "no_free_order"
                else:
                    # pulpo_order_id JETZT auf den OrderState schreiben (State
ASSIGNED) = die eine Wahrheit
                    await self._bind_order_state(db, conn.protocol_id, ref, bc,
claimed)

                    await db.commit()
            self.record_scan(conn.protocol_id, bc, now=time.monotonic())
```

`_bind_order_state` ist genau der Schreibvorgang, den heute erst LAB1 macht (OrderState anlegen/aktualisieren, `pulpo_order_id` setzen, State `ASSIGNED`) — nur nach vorn gezogen. LAB1 legt dann nichts mehr neu an, sondern findet die Bindung bereits vor.

### 2 `_claim_pulpo_order` — Reservierung aus persistierten States, Fallback raus

Den In-Memory-Guard durch eine Abfrage der persistierten aktiven States ersetzen (analog Vorlage §7.2):

```
# statt: claimed = {oid for r, oid in bound_map.items() if r != ref}
from app.modules.cmc.models import OrderState      # Pfad ggf. anpassen
reserved = (await db.execute(
    select(OrderState.pulpo_order_id).where(
        OrderState.protocol_id == protocol_id,
        OrderState.pulpo_order_id.isnot(None),
        OrderState.state.in_(("ASSIGNED", "INDUCTED", "SCANNED", "LABELED", "FAILED")),
```

```
)
)).scalars().all()
claimed = set(reserved) | {oid for r, oid in bound_map.items() if r != ref}
```

**FAILED** muss in der Liste stehen, sonst wird ein bereits gelabelter, aber in Pulpo nicht abgeschlossener Auftrag erneut gegriffen → Doppel-Sendung (Vorlage §4). Den defensiven Fallback am Funktionsende ersetzen:

```
# ALT – bindet eine bereits belegte Order an eine zweite ref → garantiert falsches
Label:
#   bound_map[ref] = rows[0].id
#   return rows[0]
# NEU:
return None          # alle Kandidaten reserviert → ENQ lehnt sauber ab (Patch 1)
```

### 3 **\_precreate\_label** — nur noch lesen, nicht claimen

```
# statt: claimed_order = await self._claim_pulpo_order(db, protocol_id, tenant_id, ref,
barcode)
claimed_order = await self._bound_order_for_ref(db, protocol_id, ref)  # liest
OrderState.pulpo_order_id
if claimed_order is None:
    logger.warning(f"PRECREATE skip: ref={ref} ohne Bindung (ENQ-Claim fehlt?)")
    return
```

Damit trifft Precreate keine Bindungsentscheidung mehr und darf gefahrlos async / nicht-awaited bleiben.

#### HINWEIS HELFER-METHODEN

Falls `_bind_order_state` und `_bound_order_for_ref` noch nicht existieren: ausformulieren.

`_bind_order_state` = `OrderState` per `(protocol_id, ref)` upserten, `pulpo_order_id` + `barcode` setzen, State `ASSIGNED`. `_bound_order_for_ref` = `OrderState` per `(protocol_id, ref)` laden und den gebundenen `PulpoPackingOrder` zurückgeben ( `None` , wenn nicht gebunden).

## 7. Edge-Cases (nicht vergessen)

---

- **Resume (Paket neu aufgelegt, `is_resume`):** erzeugt eine neue `ref` aus dem neuen Event-Zähler und claimt keinen neuen Slot — richtig so. Damit das Paket nicht ohne Bindung dasteht, die Bindung der noch aktiven Referenz auf die neue `ref` übertragen (die aktive `ref` über `tracker.is_active_barcode` → aktives Paket auflösen).
- **Nebenläufigkeit:** Falls `cmc_handle_event` -Requests parallel verarbeitet werden können (mehrere Verbindungen oder Handler-Tasks), reicht serielles Awaiting nicht. Dann Claim + Persist **atomar** machen — `SELECT ... FOR UPDATE` auf die Kandidaten oder ein Unique-Constraint auf `(protocol_id, pulpo_order_id)` für aktive States. Bei einer einzelnen, sequenziell gelesenen TCP-Verbindung nicht nötig.
- **Orphan-Label bei ACK-Reject (zu groß):** State löschen, **Bindung und Slot freigeben** (`release_cw_for_ref`), Auftrag kehrt in die Pulpo-Queue zurück (Vorlage §6). Sicherstellen, dass die nun persistierte Bindung dabei mit aufgeräumt wird.
- **END / REM:** `finalize_cw_for_ref` (END/COMPLETED) bzw. `release_cw_for_ref` (Auswurf/REM) müssen die persistierte Bindung konsistent mitführen bzw. freigeben — damit ein neu aufgelegtes Paket sauber zählt.

## 8. Testplan / Akzeptanzkriterien

---

1. **Happy Path, gleiche EANs:** 3 Aufträge A, B, C (nach `created_at`), 3 Pakete in dieser Reihenfolge scannen → Labels A, B, C in Scan-Reihenfolge. *Pflicht-Regressionstest*.
2. **Race-Test:** künstliche DB-Latenz im Precreate einbauen, Test wiederholen → bleibt korrekt (beweist, dass die Bindung nicht mehr am Precreate-Timing hängt).
3. **Überzahl:** mehr Pakete als Aufträge → das überzählige Paket erhält `result=0` (`no_free_order`), kein falsches Label.
4. **Neustart:** Paket im State `ASSIGNED / LABELED`, Service-Neustart, erneuter Scan desselben EAN → der reservierte Auftrag wird **nicht** erneut gebunden.
5. **FAILED-Guard:** Auftrag in `FAILED` → erneuter Scan wird abgelehnt (kein Doppel-Label).
6. **Reject (zu groß) bei ACK:** Bindung + Slot frei, Auftrag wieder in der CW-Liste, nächster Scan bekommt ein sauberes Label.
7. **Unit-Test `_claim_pulpo_order`:** deterministische 1:1-Zuordnung bei N gleichen EANs; bei N+1 Scans → `None` für den letzten.

## 9. Deliverables für Claude Code

---

- Code-Änderungen in `connection.py` (ENQ-Block, `_claim_pulpo_order`, `_precreate_label`) plus die Helfer aus Abschnitt 6, falls nicht vorhanden.
- `pytest` -Tests für die Akzeptanzkriterien — mindestens 1, 2, 3, 5 und 7.

- Eine Vergleichs-Markdown `docs/cmc-orderbinding-fix.md` mit: (a) Symptom, (b) Tabelle Vorher/Nachher (analog Abschnitt 4), (c) den drei Patches mit Begründung, (d) den Anti-Patterns unten — damit dieser Fehler nicht erneut entsteht.

#### ANTI-PATTERNS (IN DIE MD ÜBERNEHMEN)

- Korrektheits-relevante Bindungen **nie** in Fire-and-forget-Tasks ( `create_task` ohne `await` ) treffen — nur Nebeneffekte ohne Entscheidungscharakter gehören dorthin.
- Die Reservierungs-Wahrheit gehört in **persistenten State**, nicht in ein In-Memory-Dict (kein Neustart-Schutz, keine Sicht über Prozessgrenzen).
- Ein „defensiver Fallback“, der bei Inkonsistenz *irgendetwas* bindet, ist schlimmer als ein sauberes Reject — er verwandelt einen sichtbaren in einen stillen Fehler.
- Slot-Abbuchung und Auftragsbindung müssen **derselbe atomare Schritt** sein. Werden sie zeitlich entkoppelt, entsteht ein Fenster, in dem die Reihenfolge verrutschen kann.

---

Dieses Dokument fasst die gemeinsame Analyse zusammen und ist als Implementierungs-Briefing gedacht. Die Code-Ausschnitte sind Vorlagen — exakte Import-Pfade, Modell- und Methodennamen an das Projekt anpassen. Maßgeblich ist das Leitprinzip in Abschnitt 5.